

IMPERIAL

CHES FORTRESSES: ANALYSIS OF DRAWN POSITIONS UNDER MATERIAL IMBALANCE

Author

DANIEL RHODES

CID: 02439947

Supervised by

PROF. ASTOLFI

A Thesis submitted in fulfillment of requirements for the degree of
Bachelor of Engineering in Electrical and Electronic Engineering

Department of Electrical and Electronic Engineering
Imperial College London
2025

Abstract

Chess endgames represent one of the most theoretically rich yet computationally challenging areas of chess analysis. While strong general-purpose engines such as Stockfish excel in the middlegame, they exhibit well-documented weaknesses in endgame and fortress positions, specifically, a tendency to mis-evaluate drawn fortresses as losing due to material imbalance. No chess engine has yet been designed with fortress detection and analysis as a primary objective. This project presents a new engine, compatible with the UCI protocol, built in C++ that implements a dedicated fortress detection and evaluation system. The engine combines heuristic methods with a dynamic method based on Guid and Bratko (2012), which identifies fortresses by analysing how evaluations change across search depths. The system is evaluated against known fortress positions from master-level play and endgame tablebases, with results demonstrating both the potential and current limitations of solely heuristic-based fortress detection. This project also includes an analysis of fortress positions from theoretical studies and master-level play, comparing the strengths and weaknesses of existing chess engines.

Plain Language Summary

Chess engines are computer programs that look for the best move in a given chess position. Modern chess engines are designed to be all purpose, meaning they play well in all phases of the game. However, they are known to struggle with certain positions, especially fortresses. A fortress position arises when one player has more pieces but the opponent has built such a strong defensive setup that no progress can be made, and the game is a draw. Modern chess engines often mis-evaluate these positions, assigning a losing score to the defending side and playing as though the position is lost when it is not. This project aims to create an engine that detects and plays these types of positions more accurately than current engines. The project also analyses some fortress positions, appraising the strengths and weaknesses of the current best chess engines.

Declaration of Originality

I hereby declare that the work presented in this thesis is my own unless otherwise stated. To the best of my knowledge the work is original and all ideas and contributions from others have been appropriately referenced. In some cases, Claude Sonnet 4.6 (Anthropic) has been used to improve the clarity and flow of writing. It has not been used to generate original ideas or content. All technical work, analysis and conclusions are my own.

Copyright Declaration

Attribution-NoDerivatives Licence (CC BY-ND 4.0)

The copyright of this thesis rests with the author. Unless otherwise indicated, its contents are licensed under a Creative Commons Attribution NoDerivatives 4.0 International Licence (CC BY-ND).

Under this licence, you may copy and redistribute the material in any medium or format for both commercial and non-commercial purposes. This on the condition that; you credit the author and do not distribute modified versions of the work.

When reusing or sharing this work, ensure you make the licence terms clear to others by naming the licence and linking to the licence text.

Please seek permission from the copyright holder for uses of this work that are not included in this licence or permitted under UK Copyright Law.

Contents

Abstract	i
Plain Language Summary	ii
Declaration of Originality	iii
Copyright Declaration	iv
1 Introduction	1
1.1 Chess	1
1.2 Aims	2
1.3 Report Overview	2
2 Background	4
2.1 Overview	4
2.1.1 Why Chess?	4
2.1.2 Why Focus on Endgames?	5
2.2 Analysis of Existing Chess Tools	7
2.2.1 Search Algorithms in Chess Engines	7
2.2.2 Heuristic Evaluation	8
2.2.3 Weaknesses of Modern Engines	8
2.2.4 Endgame Tablebases	11
2.3 Software Background	11
2.3.1 Data Formats and Interoperability	11
2.3.2 Implementation Environment	12
2.4 Summary	12
3 Requirements Capture	14
3.1 Overview	14
3.2 Primary Deliverable: The Engine	14
3.2.1 Functional Requirements	15
3.2.2 Non-Functional Requirements	15

3.3	Secondary Deliverable: Fortress Analysis	15
3.4	Project Objectives and Evaluation Criteria	16
4	Analysis and Design	17
4.1	Overall Design	17
4.2	Graphical Interface: Websites vs. Offline GUIs	17
4.3	Tablebases and API Usage	18
4.4	Narrowing Scope: Focus on Fortresses	18
4.5	Supplementary Fortress Analysis	19
4.6	Summary of Design Decisions	19
5	Implementation	20
5.1	Overview	20
5.2	Algorithms	21
5.2.1	Exhaustive Search Phase	21
5.2.2	Rising Evaluation Detection	21
5.2.3	Heuristic Fortress Detection	22
5.2.4	Why This Should Outperform Conventional Engines	22
5.3	Potential Weaknesses	23
6	Test Plan and Verification	24
6.1	Overview	24
6.2	Tablebase Verification	24
6.3	Comparison with Existing Engines and Studies	25
6.4	Limitations and Honest Assessment	25
7	Results	26
7.1	Overview	26
7.2	Positions	27
7.2.1	Position One: Rook vs Queen Fortress	27
7.2.2	Position Two: The Dvoretsky Study	27
7.2.3	Position Three: Apparent Fortress with Forced Mate	28
7.3	Analysis	29

8 Evaluation	30
8.1 Project Scope and Objectives	30
8.2 Comparison with Existing Engines	30
8.3 Limitations and Honest Assessment	31
8.4 Potential and Originality	32
9 Fortress Analysis	33
9.1 Overview	33
9.2 Analyses	34
9.2.1 First Position	34
9.2.2 Second Position	35
9.2.3 Third Game	37
9.2.4 Fourth Game	37
9.2.5 Fifth Game	38
9.3 Lessons from these games	40
10 User Guide	42
10.1 Engine	42
10.2 Fortress Analysis Chapter	42
11 Glossary	44
12 Reflections	47
12.1 Ethical and Legal Matters	47
12.2 Environmental and Social Matters	48
12.3 Safety Concerns	48
12.4 Equality, Diversity and Inclusion	48
12.5 Quality Management Systems	49
Conclusions and future directions	50
12.6 Project Success and Achievements	50
12.7 Limitations	51
12.8 Reflection and Design Decisions	51
12.8.1 Key Design Choices	51
12.8.2 Most Challenging Aspects	52
12.8.3 Novel Contributions and Learning	52
12.9 Future Work	53

1

Introduction

Contents

1.1 Chess	1
1.2 Aims	2
1.3 Report Overview	2

1.1 Chess

Chess is a game of remarkable depth. While its rules are relatively simple to learn, the strategic and tactical complexity that emerges from those rules has occupied players, theorists, mathematicians, and now computer programmers for decades. Even with the advent of modern computers, chess remains a domain in which new discoveries continue to be made, not only in opening theory and middlegame tactics, but in the endgame, where paradoxically positions can be more complex than the middlegame in certain cases, despite fewer pieces being on the board. One such subtlety is the fortress, a defensive configuration in which one side has a material advantage, but the defending side has such a strong setup that they can hold a draw against what would otherwise be a losing position. Fortresses are among the most counterintuitive positions in chess. Humans and especially engines will frequently evaluate them as losing for the defending side, failing to recognise that no progress can be made against correct defence. This mis-evaluation is not merely a theoretical curiosity; it leads to genuine errors in both engine analysis and over the board play, where a defending player may not realise they were able to hold a draw.

Despite their significance, fortress positions have received comparatively little attention in the computer chess literature. Some work has been done in this area, but to the author's knowledge, no dedicated engine or analysis tool has yet been developed specifically for fortress detection. General-purpose engines such as Stockfish are optimised for tactical calculation and broad positional evaluation, and while they perform exceptionally well across most of the game, their evaluation functions are not designed to recognise the static defensive properties that characterise a fortress. The result is a gap in the landscape of chess analysis tools that this project aims to address.

1.2 Aims

This project has two principal aims. The first is to develop a chess engine specifically designed for the analysis of fortress positions. Given a position, the engine will determine whether it constitutes a fortress, or whether a breakthrough exists that allows the superior side to convert their advantage. The engine will be distributed as a UCI-compliant executable, compatible with standard chess GUIs such as Arena, requiring no specialist setup from the end user. The second aim is to analyse a few particularly interesting fortress positions, appraising modern engines' evaluations, strengths and weaknesses, and explaining the defensive mechanism, or where applicable, the breakthrough.

The potential benefits of this work are broad. Chess players seeking drawing resources in difficult endgames, coaches using engine analysis as a training tool, and researchers studying the boundary between drawn and losing positions could all make use of both the engine and the analyses.

1.3 Report Overview

The remainder of this report is structured as follows. Chapter 2 outlines the relevant background literature, covering existing research into fortress positions, the architecture of chess engines, and the role of endgame tablebases. Chapters 3, 4 and 5 detail the design, implementation, and development of the engine, discussing key design decisions and the rationale behind them. Chapters 6, 7 and 8 cover testing, results, and evaluation, assessing the engine's performance against the requirements established earlier in the report. Chapter 9 includes analyses of a few positions, explaining the weaknesses of current engines. Chapter 10 provides a user guide for the engine, and

Chapter 11 contains a glossary of technical terms. Chapter 12 concludes with reflections on the project, including ethical, social, and environmental considerations, after which conclusions and future work is outlined.

2

Background

Contents

2.1 Overview	4
2.1.1 Why Chess?	4
2.1.2 Why Focus on Endgames?	5
2.2 Analysis of Existing Chess Tools	7
2.2.1 Search Algorithms in Chess Engines	7
2.2.2 Heuristic Evaluation	8
2.2.3 Weaknesses of Modern Engines	8
2.2.4 Endgame Tablebases	11
2.3 Software Background	11
2.3.1 Data Formats and Interoperability	11
2.3.2 Implementation Environment	12
2.4 Summary	12

2.1 Overview

2.1.1 Why Chess?

The original title of this project was "Solving Puzzle Games Using Optimisation", so almost any game could have been chosen. The main challenge was finding a balance between combinatorial complexity, sufficient to justify optimisation techniques, while still having unexplored areas in the existing literature. Simple games such as noughts and crosses or Connect Four have already been solved exactly [1], while very complex games such as chess have been subject to enormous research and engineering effort, making it difficult to identify an area that has not already been extensively

studied. Chess fortresses fill this niche, combining theoretical depth with a relatively underexplored research area.

One measure of complexity is a game's state space complexity, defined as the number of legal board positions reachable during play. Noughts and crosses has around 10^5 states and Connect Four around 10^{14} , while chess has a massive 10^{43} – 10^{46} possible states. This is further compounded by the game tree complexity, which counts the number of distinct games that can be played. Claude Shannon famously estimated this number to be around 10^{120} , now known as the Shannon number [2].

Chess also has a high branching factor. From the initial position there are 20 legal moves, meaning that after one move from each player there are already 400 possible board states. After three moves from each player there are over 120 million [3]. The exponential growth of the game tree makes naive exhaustive search completely infeasible, so chess engines must rely on optimisation techniques such as pruning, move ordering, and heuristic evaluation to search the game tree efficiently [4].

Clearly it is not possible to "solve" chess in full, at least with today's technology, or possibly ever. However, chess is solvable in restricted domains with a limited number of pieces, which naturally motivates a focus on fortresses. The reduced material constrains the search space, optimisation techniques remain essential, and the quality of solutions can be measured against objective tablebase truth rather than heuristic approximation.

2.1.2 Why Focus on Endgames?

Endgames represent a particularly fascinating subdomain of chess. While openings consist largely of memorisation and middlegames require broad heuristic evaluation over many candidate moves, endgames demand a combination of memorisation, deep calculation, and precise decision making. Many endgame positions are well understood: king and pawn endings, the Lucena and Philidor rook positions, and standard checkmating patterns with the queen or rook are all examples of positions where correct technique can be learnt and applied reliably [5], [6]. In more complex positions, however, even a single error can transform a winning endgame into a draw or a loss. This unforgiving character makes endgames especially interesting to study [7].

From a computational perspective, endgames are well suited to deep searching. The smaller number of pieces reduces the average branching factor, sometimes to 10 or lower, which allows the

engine to search far deeper for a given computational budget. As a result, individual moves carry much greater weight, and the relevant forcing lines are long — which naturally favours depth-oriented search strategies over the broad heuristic evaluation used in general-purpose engines.

Modern endgame tablebases, such as the Syzygy tablebases, demonstrate both the power and the unintuitive nature of exact endgame solutions. Every position with seven or fewer pieces has been solved by retrograde analysis, working backwards from all possible checkmate positions to determine optimal play from every reachable state [8], [9]. Some of the resulting sequences are extraordinarily long and resist human explanation: the engine appears to check the king repeatedly with no apparent progress, until a win is eventually forced after hundreds of moves. Nevertheless, these sequences are provably optimal.

A key advantage of endgames for this project is that they are objectively solvable. Unlike middlegames, where engines estimate advantage through complex evaluation functions, endgame outcomes can be verified against tablebase truth [10]. This makes endgames ideal for objective optimisation, where the quality of a solution can be measured precisely rather than approximated.

Within endgames, fortresses represent a particularly compelling area of focus. They sit at the intersection of two problems: a theoretical question about the limits of material advantage, and a computational question about the limits of engine evaluation. A fortress position is one where the engine's evaluation is demonstrably wrong - it assigns a winning score to a position that is objectively drawn - and where the reason for this failure is structural rather than incidental. This makes fortresses an ideal testbed for investigating the weaknesses of heuristic evaluation, since the ground truth is known and the engine's error is reproducible and well-defined. Furthermore, fortresses are underrepresented in the existing literature relative to their practical significance: they arise in master-level play, they have caused engines to mislead both players and analysts, and yet no systematic treatment of them exists from a computational perspective. This project therefore narrows its focus from endgames in general to fortress positions specifically, using engine analysis to catalogue and characterise positions where current engines are known to fail. Endgames were also too broad for the constraints of this project.

2.2 Analysis of Existing Chess Tools

2.2.1 Search Algorithms in Chess Engines

Chess engines must navigate a game tree that is far too large for exhaustive search. A chess game can be represented as a directed tree in which each node corresponds to a legal position and each edge to a legal move. The primary computational challenge is the size of this tree, which renders complete search impossible at any meaningful depth.

The foundation of modern chess engine search is the minimax algorithm, which selects the move that maximises the minimum score the opponent can force — that is, it assumes both sides play optimally. In the absence of pruning, minimax has a time complexity of $O(b^d)$, where b is the branching factor and d is the search depth. With an average branching factor of around 35 in chess, this grows prohibitively fast [11], [12].

Alpha-beta pruning improves on minimax by eliminating branches of the game tree that cannot affect the final decision. The algorithm maintains two values, α and β , representing the minimum score the maximising player is assured of and the maximum score the minimising player is assured of respectively. Whenever $\beta \leq \alpha$, the current branch can be pruned: the opponent would never allow this line since a better alternative already exists. In the best case, with perfect move ordering, alpha-beta reduces the effective search depth by half, achieving a complexity of $O(b^{d/2})$, effectively doubling the depth reachable within a given budget [13], [14].

In practice, however, alpha-beta pruning is not used in isolation. Modern engines combine it with iterative deepening, which performs a succession of depth-limited searches of increasing depth. Results from shallower iterations are used to inform move ordering in deeper ones, since good move ordering dramatically improves pruning efficiency. Iterative deepening also provides a natural time management mechanism: the engine can return the best move found so far if time runs out mid-search.

Move ordering itself is critical. Engines typically prioritise captures, checks, and killer moves — moves that caused a cutoff at the same depth in sibling nodes earlier in the search. The null move heuristic, which tests whether passing a turn still results in a cutoff, provides additional efficiency. Together, these techniques allow engines to search far deeper than naive alpha-beta would permit [15].

A specific failure mode addressed by quiescence search is the evaluation of tactically unstable positions at the search horizon. A position that appears roughly equal may immediately deteriorate following a capture or check that lies just beyond the fixed search depth. To mitigate this, quiescence search extends the search in active positions — those with captures or checks available — until a genuinely quiet position is reached. This reduces the impact of the horizon effect in tactical positions, though it does not eliminate it entirely [12], [15].

2.2.2 Heuristic Evaluation

At the leaves of the search tree, where the search terminates without reaching a forced outcome, the engine must estimate the quality of the position using an evaluation function. This function assigns a numerical score, typically in units of centipawns, intended to capture the advantage of one side. Modern evaluation functions incorporate many features, including material balance, piece activity and mobility, pawn structure, king safety, and control of key squares [16].

Material balance is by far the most heavily weighted factor. Standard piece values - approximately one point for a pawn, three for a bishop or knight, five for a rook, and nine for a queen - form the backbone of evaluation in almost all engines. The evaluation function then adjusts this base score using positional factors, but material considerations typically dominate.

This heavy weighting of material is the root cause of many engine failures in endgame positions, and in fortress positions in particular. The evaluation function has no mechanism for recognising that a material advantage is structurally irrelevant — that is, that the opposing side's defensive setup is immune to the standard techniques for converting a material edge. As a result, engines frequently assign large positive evaluations to positions that are theoretically drawn, and continue to recommend moves that appear to press the advantage but make no real progress [17].

2.2.3 Weaknesses of Modern Engines

Several well-documented weaknesses of modern chess engines have been identified in academic literature and practical testing [18], [19].

The Horizon Effect

The horizon effect occurs when a decisive move lies just beyond the search depth, so it is not found and the position is consequently misjudged [20]. In these cases, the engine may prefer a line that merely delays an inevitable loss, or fail to recognise a winning line altogether. The effect is especially pronounced in positions involving slow positional plans, long king marches, or quiet moves with consequences far beyond the current horizon.

Although modern engines employ search extensions and late move reductions to reduce the impact of the horizon effect, it remains a fundamental limitation of depth-limited search combined with heuristic evaluation [12].

Highly Artificial Positions

Chess engines are designed to find the best practical move within a time limit, rather than to solve artificially constructed puzzles. In highly artificial positions, they may fail to find the correct continuation, particularly when it involves a king walk or an anti-positional sacrifice that contradicts standard evaluation heuristics.

An illustrative example can be found [here](#). Stockfish fails on several of these puzzles and can only solve them once a significant number of correct moves have been manually inputted. The final puzzle requires a sequence of knight underpromotions to avoid stalemate, a plan that strongly contradicts standard evaluation heuristics and is consequently missed by the engine. The full solution is available [here](#), and Stockfish's evaluation [here](#).

Fortress Detection

Fortress detection represents one of the most persistent and well-documented weaknesses of modern chess engines. A fortress is a position in which one side holds a material advantage but the opposing side has constructed a defensive setup that cannot be broken through, resulting in a theoretical draw despite the imbalance. This phenomenon is discussed in detail in the position analysis of Chapter 9, but the computational reasons for engine failure are addressed [here](#).

The root cause of engine failure in fortress positions is fundamentally evaluative rather than a simple matter of search depth. Because evaluation functions weight material so heavily, the engine assigns a positive score to the stronger side in proportion to its material advantage. In a fortress,

this advantage is real but strategically irrelevant: the defending side has no obligation to make concessions, and the attacking side has no means of forcing an entry. The evaluation function has no mechanism to detect this structural immunity and continues to produce a positive score [17].

This creates a characteristic failure pattern described by Guid and Bratko [21]: rather than the evaluation increasing monotonically as the engine finds the winning continuation, it remains approximately constant across increasing search depths, or rises only marginally. This *rising evaluation* behaviour — or rather, its absence — is the diagnostic signature of a fortress position from the engine’s perspective. A correctly functioning engine facing a genuinely winning position should produce evaluations that grow with depth as the forced win comes into view. In a fortress, no such growth occurs, yet the engine does not recognise this flat evaluation profile as evidence of a draw.

A secondary failure is practical. Even with only a modest evaluation in favour of the stronger side, the engine may continue to suggest moves that appear to press the advantage without ever making real progress. Eventually the 50-move rule intervenes and the game is drawn, but the engine’s evaluation never acknowledges this outcome. In practice, this means that a player relying on engine analysis may persist in a position that is objectively drawn, wasting time and potentially allowing the opponent counterplay.

Partial mitigations exist in current engines. Stockfish applies evaluation scaling in a small number of known drawn configurations, such as king, bishop, and knight versus lone king, or positions with opposite-coloured bishops and no pawns. However, these are hand-coded pattern recognisers covering only a small set of familiar cases. They fail entirely in novel or complex fortress structures, precisely the positions this project aims to analyse [17].

Miscellaneous Positional Difficulties

There exist cases where engines converge to a local optimum, failing to recognise a long-term strategic advantage available from another move. This can result in questionable exchanges or plans that appear sound from a short-term evaluation standpoint but fail in the longer term. This reflects the fundamental trade-off between speed and accuracy: Stockfish is designed to find strong moves quickly, whereas deeper and more exhaustive approaches play more accurately at greater computational cost [16].

In this project, the goal is not to find moves quickly but to analyse positions objectively and

deeply. This motivates design choices that favour accuracy over speed, making it possible to explore lines that general-purpose engines may deprioritise.

2.2.4 Endgame Tablebases

Tablebases are the only objective source of truth for determining the outcome of a chess endgame, with the exception of simple forced checkmates, forced repetitions, or draws by insufficient material [9]. Every chess endgame with seven or fewer pieces has been solved using the Syzygy tablebases, constructed by retrograde analysis working backwards from every possible checkmate position [8], [22]. These tablebases can be accessed [here](#).

Conventional engines such as Stockfish do incorporate tablebase lookups, displaying the correct outcome when seven or fewer pieces remain on the board. However, when faced with a position containing eight or more pieces, they revert to heuristic evaluation without making any explicit attempt to steer towards a favourable tablebase position. This project combines conventional tree search with active tablebase querying via an API, so that the engine can identify and target winning tablebase positions from positions with slightly more material.

2.3 Software Background

2.3.1 Data Formats and Interoperability

Current chess software relies on a small number of well-established data formats and communication standards to ensure interoperability between engines, interfaces, databases, and analysis tools. Adhering to these standards is essential for reproducibility and for comparison with existing engines.

The most common format for representing chess positions is Forsyth-Edwards Notation (FEN). A FEN string encodes the complete game state, including piece positions, active colour, castling rights, en passant availability, and move counters, in a compact and portable representation [23].

Move sequences are stored using Portable Game Notation (PGN), a human-readable format that supports metadata such as player names, event information, and timestamps. This is particularly useful for logging analysed games and comparing engine behaviour against published studies [24].

The Universal Chess Interface (UCI) handles communication between chess engines and user interfaces. This standardised text-based protocol allows a graphical frontend to initialise an engine, set positions via FEN, request analysis, and retrieve evaluation scores. UCI compliance allows this engine to be integrated with widely used chess platforms such as Lichess and Chess.com [25].

Tablebase data is accessed via an external API, which returns win, draw, or loss information along with the optimal moves for a given position. Compatibility with the Syzygy tablebases allows the engine's output to be verified against optimal play [26].

2.3.2 Implementation Environment

The core engine is implemented in C++, a low-level language well suited to the efficient implementation of search algorithms and board representation data structures, and one with which the author has prior experience. The UCI protocol is supported natively, allowing integration with standard chess analysis interfaces. Testing and validation are carried out through several avenues: known endgame positions with established tablebase outcomes are used to verify API functionality; endgames of eight pieces that can be converted to tablebase positions are tested for correctness; and artificial positions are used to probe edge cases including fortress positions and horizon effect failures [27].

Version control via GitHub supports incremental development and experimentation, since the codebase can be rolled back if a change degrades performance. This is particularly useful when evaluating the effect of different optimisation strategies, as versions can be compared directly.

2.4 Summary

This background chapter has presented the theoretical and computational context motivating this project. Modern chess engines achieve exceptional performance through the combination of alpha-beta pruning, iterative deepening, move ordering, quiescence search, and heuristic evaluation. However, they remain constrained by these heuristic approximations, and face well-documented failures in fortress positions, highly artificial puzzles, and cases affected by the horizon effect.

Endgames are a particularly suitable domain for addressing these weaknesses. Their reduced branching factors support deep search, their outcomes are objectively verifiable against tablebases, and the shortcomings of heuristic evaluation are most visible and consequential here. The rising

evaluation failure identified by Guid and Bratko [21] — where engines assign a persistent material advantage to a position that is in fact drawn — provides the central motivation for this project’s approach to fortress detection and analysis.

These observations motivate the aim of this project: to investigate optimisation techniques applied to chess endgames, with a focus on fortress positions where existing engines are known to fail.

Requirements Capture

Contents

3.1 Overview	14
3.2 Primary Deliverable: The Engine	14
3.2.1 Functional Requirements	15
3.2.2 Non-Functional Requirements	15
3.3 Secondary Deliverable: Fortress Analysis	15
3.4 Project Objectives and Evaluation Criteria	16

3.1 Overview

This chapter defines the requirements of the project, distinguishing between the core deliverables necessary for the project to meet its objectives and those which are desirable but not essential. These requirements were established prior to implementation and serve as the basis against which the final system is evaluated.

3.2 Primary Deliverable: The Engine

The principal deliverable of this project is a chess engine specialising in the analysis of fortress positions. The following requirements were identified:

3.2.1 Functional Requirements

The engine must:

- Accept a chess position as input, supplied via FEN string or equivalent notation
- Comply with the Universal Chess Interface (UCI) protocol, ensuring compatibility with standard chess GUIs such as Arena
- Determine whether a given position constitutes a fortress, classifying it as a draw or identifying a breakthrough for the superior side
- Output a best line, the principal variation the engine considers optimal, alongside its assessment

The engine should where possible:

- Outperform general-purpose engines on fortress positions, exploiting the advantage of domain-specific evaluation heuristics
- Provide a confidence measure or evaluation score alongside its classification, giving the user a sense of how convincingly a position is or is not a fortress

3.2.2 Non-Functional Requirements

- **Usability:** The engine should require no more setup than downloading the executable and loading it into a UCI-compatible GUI, with no additional dependencies
- **Performance:** The engine should return an assessment within a reasonable time frame for interactive analysis - on the order of seconds rather than minutes for typical positions
- **Portability:** The executable should run on standard consumer hardware without specialist software or hardware requirements

3.3 Secondary Deliverable: Fortress Analysis

In addition to the engine, an analysis of fortress positions is to be compiled as a supplementary output of the project. This resource serves both for testing during development and as a reference document of independent value. It must:

- Contain a representative set of known fortress positions
- Provide, for each position, either an explanation of the defensive mechanism that constitutes the fortress, or, where a breakthrough exists, the refutation move or line
- Discuss why current engines fail to evaluate the position correctly

3.4 Project Objectives and Evaluation Criteria

From the above requirements, the following objectives can be stated concisely:

1. Deliver a UCI-compliant executable capable of classifying fortress positions
2. Demonstrate that the engine's assessment of fortress positions is more accurate than that of a comparable general-purpose engine
3. Produce an analysis of fortress positions with accompanying annotations

Objective 2 is the primary basis for evaluation and will be assessed by testing both the engine and a reference general-purpose engine against a set of tablebase positions with known ground-truth classifications, measuring classification accuracy across both.

Analysis and Design

Contents

4.1 Overall Design	17
4.2 Graphical Interface: Websites vs. Offline GUIs	17
4.3 Tablebases and API Usage	18
4.4 Narrowing Scope: Focus on Fortresses	18
4.5 Supplementary Fortress Analysis	19
4.6 Summary of Design Decisions	19

4.1 Overall Design

The engine is designed to analyse chess positions for the presence of a fortress. Given a position as input, the engine determines whether a fortress exists or whether a breakthrough exists for the superior side. It is distributed as an executable conforming to the UCI protocol, allowing it to be loaded into any compatible GUI such as Arena for interactive analysis.

4.2 Graphical Interface: Websites vs. Offline GUIs

An early design consideration was how to present the engine to a user. The initial plan was to develop a web interface that would host the engine alongside supplementary analysis tools such as FEN loading, board visualisation, move annotation via arrows, and best-line display. However, this approach presented several practical obstacles. Building a functional, polished chess interface

from scratch in HTML and JavaScript takes a substantial engineering effort in its own right, and doing so would have diverted significant time away from the core objective: the engine itself. Upon further research, it became clear that mature, freely available GUIs such as Arena already provide most of these features for any UCI-compliant engine. This is a well-established paradigm in chess engine development — engines and interfaces are deliberately decoupled, allowing developers to focus entirely on engine logic. Adopting this approach eliminated substantial duplication of effort, removed the need to host and maintain a web server, and lowered the barrier to entry for end users, who need only download Arena and the engine executable. The trade-off is a modest reduction in customisability — Arena does not, for example, support arrow annotation directly on the board — but given the time constraints of the project, this limitation was considered acceptable.

4.3 Tablebases and API Usage

An early iteration of the engine's design centred on trading down into theoretically won endgames. Since all positions with seven or fewer pieces have been exhaustively solved and catalogued in the Syzygy tablebases, a natural strategy would be to query these tablebases once the engine had reduced the piece count to this threshold, using the tablebase verdict to guide piece exchanges toward winning positions. In practice, this approach proved unworkable. A chess engine evaluating positions at depth performs an enormous number of node evaluations per second. Issuing an external HTTP request for each tablebase lookup at this frequency is entirely impractical: network latency alone would reduce search speed by several orders of magnitude, and sustained querying of the Syzygy API at this volume would quickly exceed rate limits. The alternative — downloading a local copy of the tablebases — was equally impractical, as the full seven-piece tablebases require over 13TB of storage. The fundamental mismatch between the high-frequency demands of a search algorithm and the overhead of external data access rendered this design infeasible.

4.4 Narrowing Scope: Focus on Fortresses

The difficulty described above is symptomatic of a broader challenge: designing a general-purpose endgame engine is an unsolved and deeply complex problem. Even with tablebase access, positions with more than seven pieces cannot be analysed exhaustively, and heuristic approaches require extensive knowledge to encode reliably across a wide variety of endgame types. In response, the scope of the engine was deliberately constrained to fortress positions. This decision was motivated

by both practical and academic considerations. Practically, a narrower problem domain allows for targeted evaluation logic that can be reasoned about and tested rigorously. Academically, fortress positions represent an area that has received comparatively little attention in the computer chess literature — most engine development focuses on tactical sharpness and middlegame evaluation, leaving fortress detection as a largely open problem. By committing to this focus, the project gains depth at the expense of breadth: rather than producing a shallow general-purpose tool, it offers genuine insight into a specific and under-explored phenomenon in chess.

4.5 Supplementary Fortress Analysis

As the engine developed, it became clear that its most interesting contribution was not the engine in isolation, but what its behaviour — alongside that of existing engines — reveals about the theoretical structure of fortress positions. Accordingly, a catalogue of fortress positions was developed as a central deliverable alongside the engine, organised by material imbalance and focusing on cases where the material deficit is at least three points. This threshold reliably excludes trivial drawing cases and concentrates attention on the most instructive and theoretically rich examples. The catalogue is discussed in detail in Chapter 9.

4.6 Summary of Design Decisions

The design decisions outlined above each arose from concrete constraints encountered during development. Together, they reflect a common engineering principle: that a well-defined problem solved rigorously is more valuable than an ambitious but poorly constrained one addressed superficially.

Implementation

Contents

5.1 Overview	20
5.2 Algorithms	21
5.2.1 Exhaustive Search Phase	21
5.2.2 Rising Evaluation Detection	21
5.2.3 Heuristic Fortress Detection	22
5.2.4 Why This Should Outperform Conventional Engines	22
5.3 Potential Weaknesses	23

5.1 Overview

The engine is implemented in C++ and takes a chess position FEN as input, producing both an evaluation and a fortress detection verdict. Rather than relying solely on a single evaluation strategy, the engine combines three complementary components: an exhaustive search phase, a rising evaluation detection phase based on the method of Guid and Bratko [21], and a heuristic fallback for fortress detection adapted from Stockfish. This layered approach is designed specifically to address the weaknesses of conventional engines when evaluating fortress positions.

5.2 Algorithms

5.2.1 Exhaustive Search Phase

The first phase of the engine performs an exhaustive search over the first few moves from the input position. The purpose of this phase is to identify any immediate breakthrough attempts available to the stronger side — captures, pawn advances, or piece manoeuvres that might shatter the defensive structure before a fortress is fully established. If such a breakthrough is found within this shallow search, the position is flagged as potentially winnable and the engine reports a possible breakthrough. This phase is intentionally shallow. A full exhaustive search to any meaningful depth is computationally infeasible for positions of the complexity considered here, and the primary goal is not to solve the position but to consider any possible options that might get discarded by a traditional alpha-beta pruning method, and to rule out trivial wins that would make fortress detection irrelevant.

5.2.2 Rising Evaluation Detection

After that, the engine enters its second phase, based on the rising evaluation method described by Guid and Bratko [21]. The key insight behind this method is that in a genuinely drawn fortress position, the computer's evaluation should remain roughly stable over time: there is no path to progress, so deeper search does not reveal an increasing advantage. By contrast, in a position that merely resembles a fortress but in fact contains a breakthrough, the evaluation for the attacking side increases as the search depth increases and the engine uncovers the winning plan. The engine implements this using alpha-beta pruning over approximately ten further moves, tracking how the evaluation changes across successive depths. If the evaluation remains stable, the position is classified as a likely fortress. If the evaluation rises consistently, the position is classified as lost for the defending side despite its apparently passive nature. Alpha-beta pruning is used here rather than plain minimax for efficiency: it allows the engine to search to the required depth within a practical time budget by pruning branches that cannot influence the final result, without affecting the correctness of the evaluation.

5.2.3 Heuristic Fortress Detection

The rising evaluation method, while theoretically well-motivated, can be computationally expensive and may produce inconclusive results in highly complex positions. As a fallback, the engine incorporates a set of heuristic pattern-recognition features adapted from Stockfish's evaluation function, which includes hand-coded knowledge about drawn endgame structures. These heuristics check for features characteristic of fortress positions, for example locked pawn structures, reduced piece mobility for the stronger side, king confinement, and opposite coloured bishops. When the rising evaluation phase is inconclusive, the heuristic component provides a secondary classification, effectively acting as a safety check on the engine's overall verdict.

5.2.4 Why This Should Outperform Conventional Engines

Contemporary engines such as Stockfish, despite their extraordinary strength in conventional positions, are known to struggle with fortress detection. Their evaluations are dominated by material count and piece activity metrics, which systematically overestimate the winning chances of the materially superior side in fortress positions. A position that is a theoretical draw verified by tablebases may be assessed by Stockfish as winning by several pawns of advantage, simply because the evaluation function has no mechanism for recognising that the advantage cannot be converted. The rising evaluation method directly targets this limitation. Rather than asking how much material advantage the stronger side has, it asks whether that advantage is increasing — that is, whether deeper analysis reveals a path to progress. In a true fortress, it does not, and the engine's evaluation stabilises accordingly. This makes the approach fundamentally better suited to fortress detection than heuristic evaluation alone, since it is sensitive to the structural feature that actually defines a fortress: the inability of the stronger side to make progress. Current engines also prune away potential breakthroughs quite quickly. In a fortress position, a piece sacrifice is often needed to break through, which engines have trouble finding if the ideas run too deeply, as the lines will get discarded by the alpha-beta pruning method. Thus the exhaustive search phase is needed to ensure these sacrifices are considered. The combination of all three phases — exhaustive breakthrough search, rising evaluation detection, and heuristic fallback — is designed to be both more accurate and more robust than any single method in isolation. The exhaustive phase handles trivial wins; the rising evaluation phase handles the core fortress detection problem; and the heuristic fallback handles edge cases where the search-based methods are inconclusive.

5.3 Potential Weaknesses

A significant limitation of the current implementation is insufficient search depth. Exploring the full tree of candidate lines in complex fortress positions is computationally demanding, and the engine's current efficiency falls short of what would be required to handle these cases reliably within practical time constraints. The result is a trade-off: in more complex positions, the engine must either search shallowly and risk inaccuracy, or search deeply and become prohibitively slow. In simpler, more constrained fortress structures, where the relevant lines are fewer and the defensive mechanism or breakthrough move is easier to find, this limitation is less pronounced and the engine performs considerably better.

6

Test Plan and Verification

6

Contents

6.1 Overview	24
6.2 Tablebase Verification	24
6.3 Comparison with Existing Engines and Studies	25
6.4 Limitations and Honest Assessment	25

6.1 Overview

Testing for this project focused on verifying the engine’s ability to correctly classify fortress positions — that is, to identify drawn positions as drawn despite a significant material imbalance. Two primary verification methods were used: comparison against endgame tablebases for positions with seven or fewer pieces, and comparison against existing engines and established theoretical analysis for more complex positions.

6.2 Tablebase Verification

For positions falling within the seven-piece limit, the Syzygy endgame tablebases provide a ground truth: every such position is either a theoretical win, loss, or draw, computed exhaustively. These tablebase verdicts were used as the primary benchmark for verifying the engine’s output on simpler fortress positions. The engine was tested against several such positions and correctly identified a number of well-known elementary fortresses, including the queen versus bishop and knight fortress.

In these cases the engine's classification agreed with the tablebase result, providing confidence that the core detection logic functions correctly for positions of manageable complexity.

6.3 Comparison with Existing Engines and Studies

For positions exceeding the seven-piece tablebase limit, direct verification is not possible by tablebase lookup. Instead, the engine's evaluations were compared against those of Stockfish and against the theoretical analysis available in established sources, including Dvoretsky's Endgame Manual, the Guid and Bratko study [21], and various other literature.

The engine performed well on a subset of the tested positions. In particular, it correctly evaluated the Dvoretsky position discussed in Chapter 9 as drawn, where Stockfish assigns a significant material advantage to the stronger side and failing to find the winning move, demonstrating the intended behaviour of the rising evaluation method in a case where conventional engines are known to fail. This is analysed in more detail in chapter 9. However, the engine's performance on more complex positions was inconsistent. Positions involving multiple imbalances, deeper defensive manoeuvres, or less common fortress structures proved challenging, and the engine produced incorrect or inconclusive evaluations in several such cases. This is an expected limitation given the computational depth required to detect rising evaluations reliably in highly complex positions, and reflects the inherent difficulty of the problem rather than a fundamental flaw in the approach.

6.4 Limitations and Honest Assessment

All core components of the engine are implemented and functional. However, the system should be understood as a prototype rather than a production-strength tool. Its classification accuracy is reliable for simpler fortress structures but can fail for positions of greater complexity. Further work — in particular, extending the search depth of the rising evaluation phase and refining the heuristic fallback — would be required to improve performance on the harder cases. It is also worth noting that the positions for which the engine performs best are precisely those where conventional engines perform worst, which is consistent with the design goals of the project. The engine is not intended to be a general-purpose evaluator but a specialised tool for a narrow and well-defined class of positions.

Contents

7.1 Overview	26
7.2 Positions	27
7.2.1 Position One: Rook vs Queen Fortress	27
7.2.2 Position Two: The Dvoretsky Study	27
7.2.3 Position Three: Apparent Fortress with Forced Mate	28
7.3 Analysis	29

7.1 Overview

The engine was tested against a range of fortress positions, from simple defensive holds to positions requiring precise breakthrough detection. The results are mixed in a way that is broadly consistent with the design intentions of the engine: it performs correctly in positions where the fortress structure is clear and the defensive resource is within the breakthrough search depth, but struggles in more complex positions where the drawing mechanism is subtle or where the correct move requires sacrificing material. These results are discussed in detail below, with reference to the positions analysed in Chapter 9.

7.2 Positions

7.2.1 Position One: Rook vs Queen Fortress

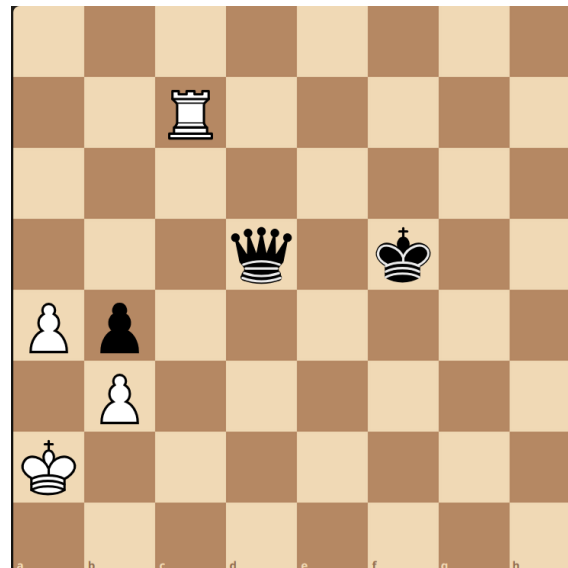


Figure 7.1: A rook versus queen fortress. The only drawing move is Rc4, which the engine correctly identifies.

In this position the engine performs correctly. There is only one drawing move, Rc4, and the engine finds it. The fortress structure here is relatively straightforward: the rook on the fourth rank holds the position by keeping the opposing king at bay and remaining within reach of its own king. The breakthrough search identifies that no pawn advance or piece sacrifice disrupts this structure, and the rising evaluation heuristic correctly flags the position as a candidate draw. Stockfish also finds Rc4 quickly, though it continues to assign a slight advantage to the stronger side rather than recognising the draw outright.

7.2.2 Position Two: The Dvoretsky Study

This position, taken from Dvoretsky's study and analysed in detail in Chapter 9, represents the most significant failure of the engine in testing. The critical moment requires the engine to avoid capturing a free queen - a move that is materially appealing but leads immediately to a drawn fortress. The engine falls into the same trap as Stockfish, taking the queen and failing to recognise that the resulting position offers no winning prospects. This is precisely the class of failure identified in the background chapter: move ordering heuristics that reward material gain direct the search

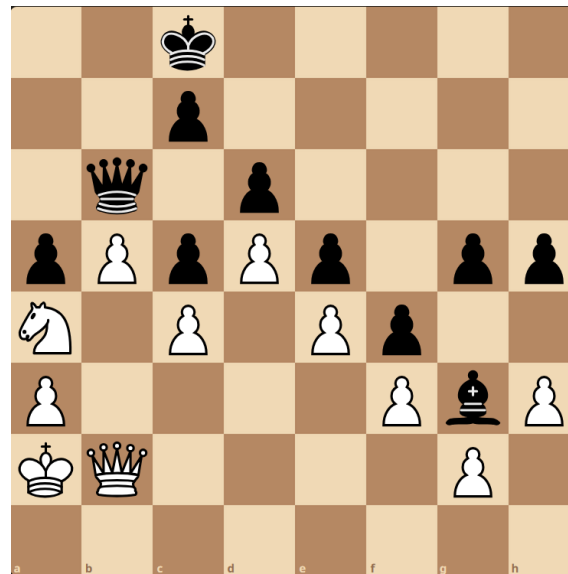


Figure 7.2: The fortress position from Dvoretsky's study, analysed in detail in Chapter 9. The engine incorrectly captures the queen, transposing into a drawn ending.

7

away from the correct continuation, and the engine does not search deeply enough in the non-capturing lines to discover that avoiding the capture is necessary. The result demonstrates a clear limitation of the current breakthrough search implementation, which does not adequately handle cases where the correct move is a deliberate refusal to gain material.

7.2.3 Position Three: Apparent Fortress with Forced Mate

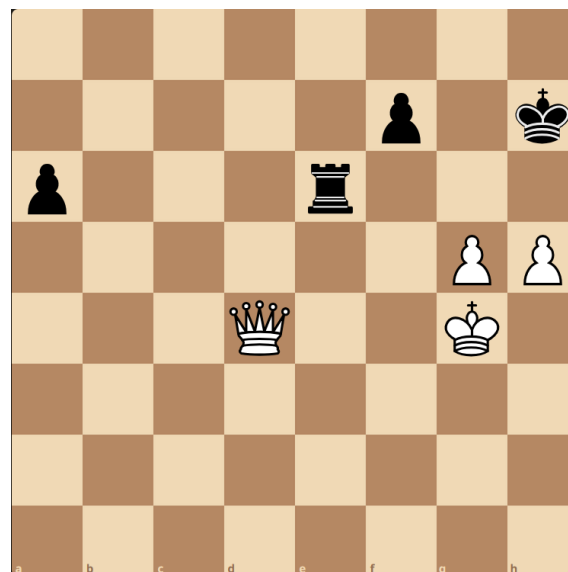


Figure 7.3: A position that superficially resembles a fortress but contains a forced checkmate sequence.

This position is instructive precisely because it is not a fortress, despite appearing to be one at first glance. The defensive structure looks plausible, but a forced checkmate sequence exists and is found by the engine without difficulty. This result is encouraging for two reasons. First, it confirms that the engine does not over-apply the fortress heuristic: it does not flag every position with a flat evaluation profile as a draw. Second, it demonstrates that the breakthrough search is functioning correctly in positions where a decisive line does exist — the engine finds the forced mate and assigns the correct evaluation rather than misidentifying the position as drawn.

7.3 Analysis

The three positions presented here are representative of the broader pattern observed across all positions tested. The engine performs reliably in positions where the fortress structure is unambiguous and the defensive resource is a straightforward move within the search depth. It also correctly handles positions that superficially resemble fortresses but contain a forced winning line, suggesting that the detection heuristic does not produce an unacceptable rate of false positives.

The significant failure case is the Dvoretzky position, where the correct move requires refusing a material gain. This points to the most pressing limitation of the current implementation: the breakthrough search and move ordering heuristics are not well equipped to handle positions where the winning resource for the defending side is a deliberately passive or materially unfavourable move. Addressing this would require either a more sophisticated move generation strategy that explicitly considers non-capturing alternatives in fortress candidate positions, or a richer evaluation function capable of recognising that material gain is locally appealing but globally counterproductive.

It should also be noted that the engine's performance on the positions in this chapter is broadly consistent with its performance relative to Stockfish: in positions where Stockfish struggles, the engine tends to struggle also, while in positions where the correct continuation is within reach of standard search techniques, both engines perform correctly. The value of the engine therefore lies less in outperforming Stockfish on individual positions and more in providing a framework, the rising evaluation diagnostic combined with targeted breakthrough search, that can identify and characterise fortress failures in a principled way.

8

Evaluation

Contents

8.1 Project Scope and Objectives	30
8.2 Comparison with Existing Engines	30
8.3 Limitations and Honest Assessment	31
8.4 Potential and Originality	32

8.1 Project Scope and Objectives

The original objective of this project was to develop an engine capable of solving puzzle-like chess positions through optimisation, with a particular focus on fortress detection. As described in the Analysis and Design chapter, the scope evolved considerably from this starting point — moving from a web-based general-purpose solver to a specialised C++ fortress detection engine accompanied by an analytical catalogue. This evaluation assesses how well the final deliverables meet the spirit of those original objectives and how they compare to existing work in the field.

8.2 Comparison with Existing Engines

The engine's approach differs fundamentally from that of conventional engines such as Stockfish. Stockfish evaluates positions primarily through material counting and heuristic assessment of piece

activity, pawn structure, and king safety. These metrics are highly effective across the vast majority of chess positions, but are structurally poorly suited to fortress detection: a fortress, by definition, trades piece activity and sometimes material for structural resilience, and no heuristic weighting of conventional features can reliably capture this. The rising evaluation method used here targets the problem differently, asking not how advantageous a position looks statically but whether that advantage is increasing as search depth grows. This is a more appropriate question for fortress detection, and in well-defined cases the engine correctly identifies positions that Stockfish miscalculates. The queen versus bishop and knight fortress, and the Dvoretsky position discussed in Chapter 9, are both correctly classified as drawn by the engine where Stockfish assigns a substantial winning advantage to the stronger side. It must be acknowledged, however, that the

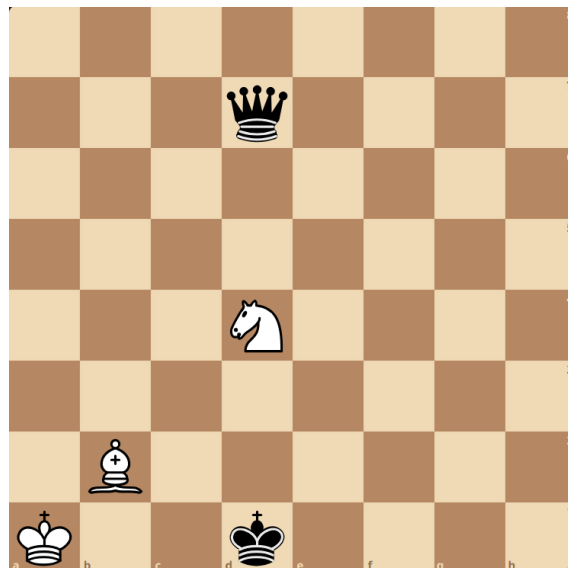


Figure 8.1: A fortress of a knight and bishop against a queen.

engine’s superiority over Stockfish is narrow and context-dependent. Stockfish’s overall strength vastly exceeds that of this engine, and its fortress detection has improved in recent versions through handcrafted scaling factors. The contribution here is not to replace Stockfish but to demonstrate that a search-based rising evaluation approach can succeed on cases where heuristic evaluation fails — and to provide a framework that could, in principle, be extended and refined.

8.3 Limitations and Honest Assessment

The engine’s most significant practical limitation is computational efficiency. The rising evaluation method requires searching to sufficient depth to determine whether an evaluation is genuinely stable

or gradually worsening, and in complex positions this depth requirement becomes prohibitive. As the engine is pushed toward harder positions — those with more pieces, less stereotyped structures, or longer defensive manoeuvres — search times increase rapidly and evaluation accuracy degrades. This is not a flaw in the underlying approach so much as a reflection of the intrinsic difficulty of the problem: the positions where fortress detection matters most are precisely those where search is most expensive. A secondary limitation is accuracy on complex positions. While performance on simpler, well-defined fortresses is encouraging, the engine produces inconsistent results on positions requiring deeper or less pattern-like defensive play. The heuristic fallback, adapted from Stockfish, partially mitigates this but introduces the same dependence on hand-coded evaluation knowledge that the rising evaluation method was designed to transcend.

8.4 Potential and Originality

Despite these limitations, the approach has genuine potential. To the author's knowledge, a dedicated fortress detection engine combining exhaustive breakthrough search, rising evaluation detection, and heuristic fallback has not been previously implemented as a standalone system. Existing work in this area, including the Guid and Bratko study [21], has demonstrated the theoretical validity of rising evaluation as a detection method, but a practical implementation targeting the specific class of material-imbalanced fortresses considered here represents a novel contribution. With further development — in particular, more efficient search through improved move ordering and pruning, and a larger set of heuristic patterns for the fallback component — the engine could plausibly be extended to handle more complex positions reliably. The catalogue developed alongside it also provides a concrete benchmark against which future improvements could be measured, which is itself a contribution the field currently lacks.

9

Fortress Analysis

Contents

9.1 Overview	33
9.2 Analyses	34
9.2.1 First Position	34
9.2.2 Second Position	35
9.2.3 Third Game	37
9.2.4 Fourth Game	37
9.2.5 Fifth Game	38
9.3 Lessons from these games	40

9.1 Overview

This chapter presents an analysis of a selection of fortress positions, chosen for their theoretical significance and the particular difficulties they pose for contemporary chess engines. A fortress, broadly speaking, is a defensive configuration in which the materially inferior side achieves a draw not through active counterplay but by constructing an impenetrable zone that the opponent cannot break through. While such positions are well understood by human analysts — and in many cases verified as theoretical draws by endgame tablebases — they represent a persistent blind spot for engine evaluation. Engines tend to assess positions based on material count and piece activity; a fortress, almost by definition, sacrifices both in exchange for structural resilience, which standard evaluation functions are poorly equipped to recognise.

The positions examined here have been selected to illustrate this tension across a range of material imbalances. For each position, the engine's assessment is considered alongside a deeper

analysis of the defensive mechanism at work, with the aim of showing not only that the position is drawn, but why the engine fails to see it.

The user whiteshark on chessgames.com has compiled a more comprehensive list of fortresses from master-level play, which can be found [here](#).

9.2 Analyses

9.2.1 First Position

[Click here](#) to view the lichess analysis board.

This game comes from Arshak Petrosian vs Laszlo Hazai from the 6th International Junior Tournament (1970).

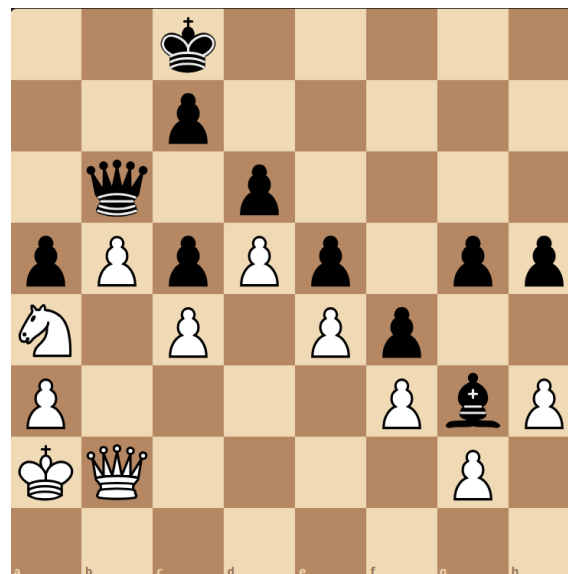


Figure 9.1: A fortress of a knight and bishop against a queen.

This position, examined in Dvoretsky’s Endgame Manual and analysed by Guid and Bratko [21], illustrates a particularly instructive failure mode for chess engines. The material situation gives White a seemingly decisive advantage, and historically, virtually all engines responded to it the same way: capturing the queen on b6 with the knight. This is superficially logical — White wins a queen for a knight — but fatally shortsighted. After 1...cxb6, Black creates an impregnable fortress. The pawns on h4 and g4 lock the kingside completely, and White’s extra material cannot be brought to bear; even sacrificing the queen for the bishop fails to break the

position down. Modern engines have partially corrected this: stronger contemporary engines do recognise the resulting position as drawn and no longer list the queen capture among their top choices. However, this represents only a partial improvement. The engines avoid the blunder but cannot identify the winning plan, which requires a long sequence of subtle manoeuvres: Qd2, followed by transferring the king to a4 and the knight to b3, from where it can capture the a5-pawn and transition to a winning endgame. This kind of deep, slow regrouping — involving no immediate material gain and no obvious piece activity — lies outside what standard evaluation functions reward, exposing a fundamental limitation in how engines assess positional long-term plans. If one analyses the position with the multiple lines turned up to 5, Qd2 does appear as the engine's fourth choice, but the engine's line from the starting position still includes the blunder as previously. Only after letting the engine analyse after going a few moves forward does it actually find the winning plan. (to note - the engine actually finds an alternative win by sacrificing the knight for one of the centre pawns - this also works but can only be found by letting the engine analyse after 3 or 4 moves.)

9.2.2 Second Position

This position arises from the game between Eric Rosen and Chen Wang from the 2023 Biel Chess Festival. The full game can be viewed [here](#). This is one of the most complex positions in this anal-



Figure 9.2: The fortress position from Rosen vs Wang, Biel 2023. White is down a rook for a knight down but holds a theoretically drawn fortress.

ysis, and arguably the most instructive from an engine evaluation perspective. White is materially

a rook down for a knight, a substantial deficit by any standard evaluation, yet has constructed a fortress that proves impossible to break in practice. The knight on b5 is the defensive cornerstone: it permanently blockades the b-file, neutralises Black's most active piece placement, and cannot be profitably exchanged. If Black trades the bishop for the knight on b5, White recaptures with fxg5, generating two connected passed pawns and sufficient counterplay that the engine actually evaluates the resulting position as better for White. The pawn on a4 is well defended and the overall pawn structure is sufficiently closed to deny Black any meaningful entry point. Browser-based versions of Stockfish evaluate this position with a meaningful advantage for Black, and stronger versions are only marginally less optimistic, yet they too fail to convert. What is particularly revealing is the nature of the winning attempt that Stockfish itself pursues when set to play against another engine. The first idea is the pawn sacrifice b3, opening the long diagonal for the dark-squared bishop and granting it scope it would otherwise lack. The more significant breakthrough attempt involves extensive piece shuffling to place all pieces on their optimal squares, followed eventually by f5, sacrificing the light-squared bishop after exf5 or gxf5 with Bxf5. The rooks are repositioned to the f-file in advance to support this break. The resulting position after the sacrifice transforms into a rook versus two knights imbalance, a materially complex situation where Black retains practical chances, though a Stockfish versus Stockfish game run from the resulting position ended in a draw. A particularly spectacular variation arises after 1...f5 2.gxf5 Bxf5 3.exf5 Rxf5 4.Bd1 e4 5.dxe4 Qxe4 6.fxe4 Rxf2 7.Bg4 Bd2 8.Qd1 Be3 9.Kh1 Rf1+ 10.Qxf1 Rxf1+, a queen sacrifice line that leads to a position evaluated at 0.00 - Black can force a perpetual check and no more. The engine generates extraordinarily complex play in the attempt to breach the fortress, yet the defensive resources available to White, while perhaps beyond human precision, are always sufficient. This illustrates the central difficulty of the position: the fortress holds not because no winning attempt exists, but because every winning attempt, however ingenious, can be met with precise defence. From an engine evaluation standpoint, this position is the clearest example in this catalogue of the gap between evaluation and convertibility. Stockfish is not wrong to identify Black as having the better position in a general sense - the material advantage is real and the winning attempts are dangerous. What the engine cannot determine is that these attempts are ultimately insufficient, and that the defensive resources available to White are always adequate provided the correct responses are found. The flat evaluation profile described by Guid and Bratko [21] is less visible here than in simpler fortresses, precisely because the position is very complex rather than structurally trivial. This makes it the hardest class of fortress for any detection method to handle: the engine's confidence is partially justified, the winning attempts are real, and the draw only becomes apparent after exhaustive analysis of every possible breakthrough.

9.2.3 Third Game

This position arises from Yuri Averbakh vs Igor Bondarevsky, 1948 USSR Championship. [Click here](#) to view the analysis board.

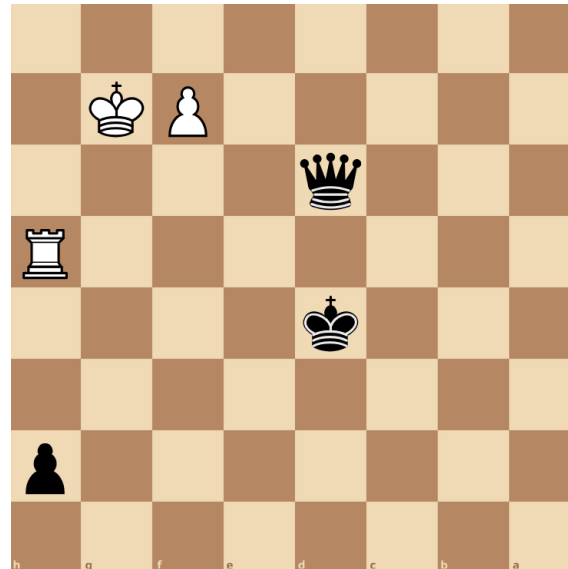


Figure 9.3: The fortress from Averbakh vs Bondarevsky.

On move 60, following $Rxh4$, the engine confirms that Black has a forced win, though it is far from trivial to find over the board — and indeed, Bondarevsky did not find it. After $60...Ke6$ $61.Rh3$, the position becomes a fortress. The key defensive principle is that the rook must reach the third rank: from there it can block checks from the queen and remain within reach of the king or pawn, ensuring it is always protected. Should the rook fail to reach the third rank, Black can force a fork and win it. Conversely, once it arrives there, Black has no way to make progress. What makes this position particularly instructive from an engine analysis perspective is Stockfish's evaluation after $61.Rh3$: it returns -0.8 in Black's favour, despite the position being a theoretical draw. Following the engine's top line, the evaluation even climbs to -1.0 at certain points. This again illustrates the core weakness that motivates this project: engines remain too materialistic to recognise a fortress at the evaluation level, even when they are capable of playing the defensive moves accurately when given the chance.

9.2.4 Fourth Game

[Click here](#) to view the analysis board.

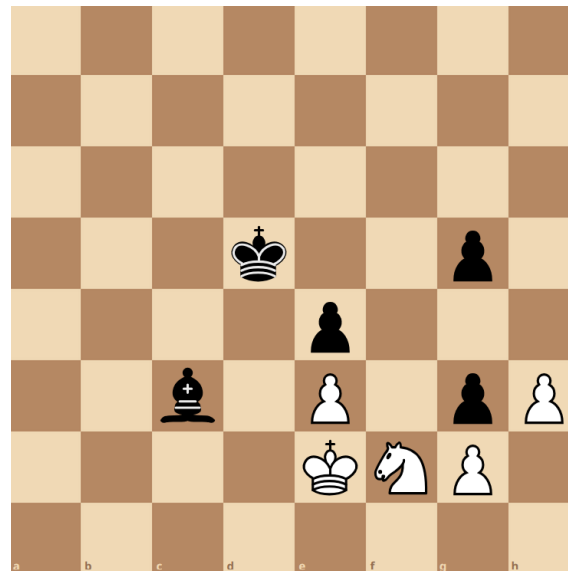


Figure 9.4: The fortress from Nakamura vs Serper.

This position arises from Hikaru Nakamura vs Grigory Serper, 2004. On move 82, Serper plays the spectacular sacrifice $Nxe4$, surrendering the knight in full knowledge that the resulting position is an inescapable fortress. The logic is precise: after the knight is removed, White's remaining pawn on the kingside cannot be supported or advanced, and the bishop is the wrong colour to attack either of Black's remaining pawns. The Black pawns on the edge of the board offer no winning chances either, since the White king is perfectly placed in front of them, nullifying any promotion threat. The position is a theoretical draw by insufficient mating material and an inability to make progress on either side of the board. It is worth noting that $Nxe4$ is not the only drawing move in this position - the engine suggests $Ng4$, which also holds. What makes Serper's choice striking is its clarity: rather than finding an adequate defensive move, he simplifies the position into one where the draw is structurally obvious. Despite this, Stockfish evaluates the resulting position at approximately -0.5 in Black's favour, once again failing to recognise a dead draw. This is a further illustration of the engine's inability to account for the fortress structure: Black is up a bishop, so the engine thinks there is some advantage, yet even this small negative score is entirely wrong - the position offers neither side any winning prospects whatsoever.

9.2.5 Fifth Game

This position arises from the game between Vladimir Kramnik and Hao Wang, played in 2012. A link to the game can be found [here](#).

From this position, White has a winning advantage, but the conversion requires precise play.

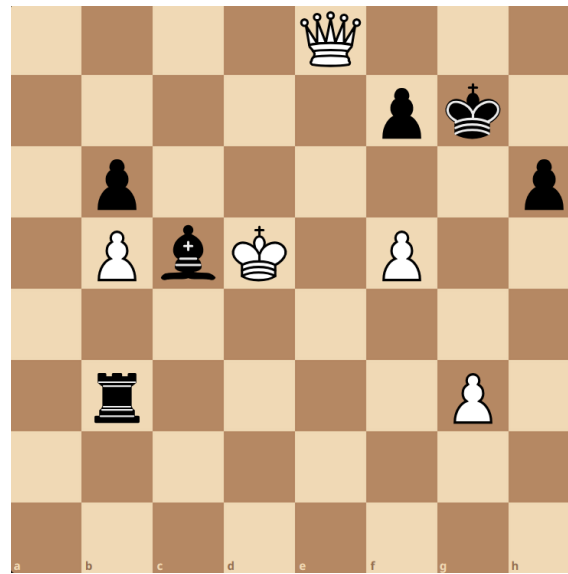


Figure 9.5: The position from Kramnik–Wang, 2012. White holds a winning advantage but the correct continuation is difficult to find.

Rather than advancing the f-pawn, White needed to find a different plan to make progress. Kramnik instead pushed the f-pawn, and this turns out to be the critical error: it allows Black to reorganise the defensive pieces into a configuration that proves impenetrable. This is a good illustration of how fortress positions can arise not from a single blunder but from a subtly incorrect plan - the f-pawn advance is not obviously wrong, and without deep calculation it is easy to miss that it hands Black precisely the defensive structure needed.



Figure 9.6: The resulting fortress. Black’s bishop permanently defends the b-pawn, and the rook and pawns provide a shelter the White queen cannot penetrate.

Once the fortress is established, White has no winning chances. The b-pawn is permanently

defended by the bishop. Black's rook and pawns create a shelter for the king that the White queen alone cannot breach - without a second piece to coordinate with, the queen has no way to simultaneously threaten the king and attack the defensive pawns. The position will eventually be drawn by the fifty-move rule or repetition. This position is particularly interesting from an engine evaluation perspective. The first diagram represents exactly the kind of position where engines can mislead: the advantage is real and large, but converting it requires finding a plan that avoids a specific structural concession many moves in advance. An engine evaluating at insufficient depth will not see that the f-pawn advance leads to a dead draw, and may even suggest it as a reasonable plan given that it creates a passed pawn. The second diagram, by contrast, is a case where stronger versions of Stockfish do eventually recognise the draw, though browser-based versions continue to assign White a meaningful advantage. The transition between these two positions — from winning to drawn — was determined by a single strategic decision, which is precisely what makes this game a compelling case study in both practical endgame play and the limits of engine analysis.

9.3 Lessons from these games

Several consistent themes emerge from the positions analysed in this chapter, each illustrating a distinct dimension of engine failure in fortress positions.

The most fundamental is that engines are systematically materialistic. In every position examined, Stockfish assigns a non-trivial advantage to the side with more material, even when the position is a theoretical draw. This is not a rounding error or a depth limitation — it reflects a structural bias in the evaluation function, which has no mechanism for recognising that a material advantage can be strategically irrelevant. The evaluation hovers persistently above zero without ever resolving, which is precisely the flat evaluation profile that Guid and Bratko identify as the hallmark of a misidentified fortress [21].

A second failure follows directly from this. Because the engine believes the stronger side is winning, it sometimes cannot find the breakthrough move. In several of the positions analysed, the correct defensive move is unintuitive - a piece sacrifice, not capturing a free piece, or a quiet move that establishes the defensive structure rather than making material gains. The engine does not prioritise or discards these moves precisely because they appear to worsen the material balance, which the evaluation function treats as a proxy for overall advantage. The result is that the engine fails to find the drawing resource not because the search depth is insufficient, but because the move

ordering and pruning strategies direct it away from the correct line.

A third and subtler failure concerns the engine's inability to reason about the stability of a defensive setup. Once a fortress is established, the relevant question is not "What is the best move from this position?", but "Does any sequence of moves exist that breaks the fortress?". These are fundamentally different questions. The former is local and is what tree search is designed to answer; the latter is global and requires reasoning about the complete set of attacking options across all lines. No finite search depth can answer this question exhaustively for positions above the tablebase boundary, and the engine has no alternative mechanism to fill this gap. It therefore continues to treat the position as a search problem with a solution yet to be found, rather than as a draw with no solution available.

Taken together, these failures suggest that the limitations of engines in fortress positions are not incidental or easily patched. They are a consequence of the core architecture: material-weighted evaluation combined with move-by-move tree search is well suited to converting advantages in open, dynamic positions, but is poorly matched to the static, structural nature of fortress defence. Addressing this requires either richer evaluation functions that can represent structural sufficiency, or search strategies that explicitly test for the absence of a breakthrough rather than merely failing to find one.

10

User Guide

Contents

10.1 Engine	42
10.2 Fortress Analysis Chapter	42

10.1 Engine

The fortress detection engine is provided as a compiled C++ executable, available in the project repository (link it). As a standard executable, it is compatible with any chess analysis platform that supports external engine integration via the UCI protocol, such as Arena or Cutechess. To use the engine, it should be loaded as an external engine within the chosen platform in the usual way; no additional configuration is required for basic use. It should be noted that the engine is specialised for fortress detection rather than general play. It is designed to be used on positions where a significant material imbalance already exists, and its evaluations are most meaningful in that context. Using it on positions outside this scope — for instance, early middlegame positions — may produce unreliable results.

10.2 Fortress Analysis Chapter

The fortress analysis chapter is intended to serve as both a reference catalogue and a practical guide. Each position includes an explanation of the defensive mechanism at work and a discussion of the

strongest attempts to break the fortress down, along with an appraisal of the current best engines' weaknesses. At the end of the chapter, a summary of common themes is provided. This outlines recurring structural and tactical patterns that appear across fortress positions of different types — both from the perspective of the defending side, identifying the key principles for constructing and maintaining a fortress, and from the perspective of the attacking side, identifying the most reliable methods for finding breakthroughs when they exist. This summary is intended to be useful as a standalone reference for players who wish to apply fortress ideas in their own games without working through every individual position.

11

Glossary

This is intended to explain chess specific terms or other technical terms which the reader may be unfamiliar with. For an explanation of the rules of chess, [click here](#).

- **50 Move Rule:** If 50 moves have been made by both players without a capture or pawn move, the game results in a draw. This is intended to ensure that two players who endlessly shuffle their pieces around with no checkmate will reach a draw, but this has unfortunate implications for computer chess, as computers are able to see more than 50 moves ahead.
- **Blessed loss:** Where there is a forced win for the opponent but doing so would require more than 50 moves with optimal play, resulting in a draw by the 50 move rule.
- **Distance to Mate (DTM):** The number of moves required to reach checkmate from optimal play. Note that DTM and DTZ use half-moves. The 50 move rule is counted in full moves.
- **Distance to Zero (DTZ):** The number of moves required to reach a zeroing move, i.e. a pawn move or capture (or checkmate) that would reset the 50 move rule counter.
- **Endgame:** The last phase of the game, where few pieces are left on the board. This can be defined in different ways, but Lichess defines it as when there are 6 or fewer queens, rooks, knights or bishops left on the board.
- **Evaluation Function:** This is what chess engines use to evaluate a position. They take into account material, positional advantage, and other things to calculate a value. Most chess engines denote for example +2 for a decisive advantage for white, -8 for a crushing advantage

for black, and use a hash symbol with a number to denote a forced checkmate sequence. $+/-0$ is a draw.

- **FIDE:** The International Chess Federation. It is the governing body of the sport of chess, and it regulates all international chess competitions, setting the rules for play.
- **Forsyth-Edwards Notation (FEN):** Human readable notation that can be entered into a computer to display the desired position.
- **Frustrated Win:** Where there is a forced win for the player but doing so would require more than 50 moves with optimal play, resulting in a draw by the 50 move rule.
- **Fortress:** A position that arises when one side has a material advantage but cannot force a win due to the opponent having a strong defensive setup.
- **Horizon Effect:** When there is a winning line but it is just beyond the search depth so the engine does not find it and plays incorrectly.
- **Material:** The amount of points each piece is worth.
 - A Queen is worth 9 points
 - Rooks are worth 5 points
 - Knights and Bishops are worth 3 points
 - Pawns are worth one point, although they become more valuable the further up the board they are

A side is said to have a material advantage if they possess more points of material than the opponent.

- **Perfect / Optimal Play:** It is assumed throughout this project that the computers will always make the move that, for the winning side, reaches checkmate the fastest, and for the losing side, delays mate for the longest. If the position is a draw it is assumed that neither side will blunder into a forced checkmate sequence.
- **Portable Game Notation (PGN):** A standard human-readable format for recording chess games.
- **Retrograde Analysis:** A method of analysis used to create endgame tablebases, done by backwards induction from known outcomes (win, draw or loss).

- **Tablebase:** A computerised database containing precalculated evaluations of endgame positions.
- **Search Depth:** The number of moves ahead the engine is searching.
- **Universal Chess Interface (UCI):** Software that allows chess engines to communicate with a user interface.
- **Zugzwang:** A position in which the player would wish to pass their turn to keep a good position, but the obligation to move forces them to worsen their position.

12

Reflections

Contents

12.1 Ethical and Legal Matters	47
12.2 Environmental and Social Matters	48
12.3 Safety Concerns	48
12.4 Equality, Diversity and Inclusion	48
12.5 Quality Management Systems	49

12.1 Ethical and Legal Matters

This project makes use of open-source software and datasets, including code from Lichess and the Syzygy endgame tablebases. All such resources will be used in compliance with their respective licences, and no proprietary or unlicensed material will be incorporated. Care will be taken throughout to clearly distinguish original contributions from existing tools, engines, or datasets, all of which will be properly cited.

The project raises no significant ethical concerns, as it is intended purely as an analytical and educational tool. A potential concern is the use of chess engines to gain an unfair advantage in competitive play; however, this issue is not specific to this project and is already addressed by established anti-cheating regulations set out by governing bodies such as FIDE. No additional mitigation is therefore required on this project's part.

12.2 Environmental and Social Matters

The primary environmental consideration is energy consumption. Chess engines and endgame tablebase computation can be computationally intensive; where possible, pre-computed tablebases will be used rather than performing redundant calculation, reducing unnecessary energy usage.

From a social perspective, if the engine successfully identifies errors in previously accepted fortress theory, this could challenge existing published analysis. However, the revision of established theory in light of new evidence is a normal and constructive part of the development of chess knowledge, and any such findings would be presented carefully and with appropriate context. On balance, the project is more likely to have a positive social impact by contributing a new analytical tool for the study of endgame positions.

12.3 Safety Concerns

The primary safety consideration is the risk of prolonged screen exposure associated with extended software development work. This will be mitigated by taking regular breaks in accordance with standard ergonomic guidance, and by managing workload to avoid excessive or unsustainable working patterns.

12.4 Equality, Diversity and Inclusion

The engine will be distributed as a UCI-compliant executable compatible with standard chess GUIs, many of which offer built-in accessibility features such as font and text size customisation and adjustable colour schemes. Where the choice of GUI permits, these features will be noted in the documentation accompanying the engine. The engine is designed to run on commonly available consumer hardware, deliberately avoiding a dependency on high-performance or specialist computing resources. This lowers the barrier to access for users who do not have access to high-specification machines.

12.5 Quality Management Systems

A number of quality management practices will be employed throughout the project to ensure correctness, maintainability, and reproducibility of results.

Version Control: All source code will be maintained under Git version control, with commits made at regular intervals corresponding to meaningful development milestones. This provides a full history of changes, supports rollback in the event of regressions, and ensures the codebase is recoverable.

Testing: The engine will be validated against a curated set of positions with known ground-truth classifications — positions confirmed to be fortresses or confirmed to have a breakthrough — drawn from established chess literature and the Syzygy tablebases where applicable. Classification accuracy on this test set will form the primary basis for evaluating the engine’s correctness, as outlined in the Requirements Capture chapter.

Benchmarking Against a Baseline: Engine performance will be compared against a general-purpose engine (such as Stockfish) on the same test positions. This provides a meaningful reference point and guards against the possibility that any apparent success is attributable to factors unrelated to the fortress-specific logic.

Code Quality: Code will be written with clarity and modularity in mind, with logical separation between the search algorithm, evaluation function, and fortress-specific heuristics. This separation makes individual components easier to test in isolation and simplifies future modification.

Conclusions and future directions

Contents

12.6 Project Success and Achievements	50
12.7 Limitations	51
12.8 Reflection and Design Decisions	51
12.8.1 Key Design Choices	51
12.8.2 Most Challenging Aspects	52
12.8.3 Novel Contributions and Learning	52
12.9 Future Work	53

12.6 Project Success and Achievements

This project set out to investigate fortress positions in chess. These are cases where one side holds a material advantage that cannot be converted to a win. The project had two complementary aims: to assemble a structured analysis of such positions, and to develop an engine capable of detecting and correctly evaluating them. Both aims have been substantially achieved, though with limitations that are worth acknowledging honestly.

The primary deliverable, the engine, is described in Chapter 5. The three-phase approach - exhaustive breakthrough search, rising evaluation detection, and heuristic fallback adapted from Stockfish - handles simpler fortress positions correctly and provides a working proof of concept for a detection strategy that goes beyond material-weighted evaluation. In straightforward cases such as the queen versus bishop and knight fortress and the Dvoretzky position, the engine identifies the draw where Stockfish does not. That this was achieved within the constraints of a single-developer project in a limited timespan is, in itself, a worthwhile result.

The supplementary analysis of fortress positions is presented in Chapter 9. Across a range of material imbalances, the appraisal documents positions from master-level play where modern engines fail, characterises the nature of each failure, and identifies the precise structural features that the engine is unable to account for. This represents a more systematic treatment of fortress positions than currently exists in the literature, where such positions are typically discussed in

isolation as curiosities rather than as a coherent class of engine failure. The catalogue also demonstrates, concretely and repeatedly, the flat evaluation profile identified by Guid and Bratko [21]: in every position examined, Stockfish assigns a persistent material advantage to the stronger side at evaluations that neither grow with depth nor resolve to a winning line, confirming that the failure is systematic rather than incidental.

12.7 Limitations

Transparency requires acknowledging where the project falls short. The fortress analysis, while structured and analytically substantive, covers only a small number of positions. There are vast number of possible fortresses that remain unanalysed, and the positions selected necessarily reflect the author’s judgement about what is most representative and interesting rather than any exhaustive survey.

The engine similarly has clear limitations. In more complex fortress positions - those with greater piece counts, more complex pawn structures, or defensive resources that lie beyond the breakthrough search depth - it fails to improve on Stockfish. The rising evaluation heuristic is a useful diagnostic but is not a proof of fortress status: it can produce false positives in positions that are genuinely winning but require very deep search to confirm. These limitations are a consequence of the restricted scope of the project rather than fundamental flaws in the approach, and they point clearly towards future directions.

12.8 Reflection and Design Decisions

12.8.1 Key Design Choices

Several significant design decisions shaped the project’s direction. The most consequential was the early choice to include a catalogue of fortress positions rather focussing solely on the engine. An earlier framing of the project included only the engine as the deliverable. Including the fortress analysis produced a more coherent project with a clearer contribution, and acknowledged honestly that a single-developer engine was unlikely to surpass Stockfish in general-purpose play.

A second important decision was the choice to focus exclusively on positions with a material imbalance of at least three points. This threshold excludes trivial cases such as single-pawn-up

endings, which are often drawn but do not exhibit the structural properties that make fortress positions computationally interesting. It also excludes categories such as opposite-coloured bishop endings, which are theoretically rich but form a narrow and well-understood special case. The threshold was set deliberately to keep the focus on positions where the material advantage is large enough to be strategically significant.

The decision to implement the engine in C++ with UCI compliance was also significant. UCI compliance allowed the engine to be tested directly against Stockfish within standard chess interfaces, making comparisons clean and reproducible. C++ imposed a steeper implementation overhead than a higher-level language would have, but provided the performance necessary for deep search within reasonable time limits.

12.8.2 Most Challenging Aspects

The most technically challenging aspect of the project was the design of the breakthrough search. The core difficulty is definitional: a breakthrough is not a single type of move but any move or sequence that disrupts the defensive structure, which varies considerably between fortress types. Designing a search that is general enough to cover the range of positions in the catalogue, yet specific enough to avoid searching irrelevant lines, required significant iteration. The eventual approach — searching for pawn advances, piece sacrifices, and king marches within a fixed depth, then falling back to rising evaluation detection if no breakthrough is found — is a reasonable approximation but not a complete solution, and refining it further is the most promising direction for future work.

A subtler difficulty was the absence of ground truth in positions above the seven-piece tablebase boundary. For positions within the boundary, the Syzygy tablebases provide an exact verdict and the engine's output can be verified objectively. For positions with more pieces, the only available evidence is the engine's own evaluation and the historical record of master-level play. This made it harder to be certain that a position identified as a fortress by the rising evaluation heuristic was genuinely drawn, rather than merely beyond the current search depth.

12.8.3 Novel Contributions and Learning

The most genuinely novel aspect of this project is the application of the Guid and Bratko rising evaluation framework as a practical detection mechanism across a structured catalogue of positions.

Previous work on rising evaluation has been largely theoretical; applying it systematically to real positions from master-level play, and using it to characterise engine failure across a range of material imbalances, is a contribution that has not, to the author’s knowledge, been made before.

Professionally, the project developed skills in research scoping and prioritisation that are perhaps less visible than technical contributions but equally important. The original project scope was substantially broader than what was ultimately delivered, and the process of narrowing the focus to what was achievable and meaningful within the available time required repeated judgement calls.

12.9 Future Work

The most immediate direction for future work is the expansion of the catalogue. A systematic method for generating candidate fortress positions, for example, by searching large game databases for positions where Stockfish’s evaluation is persistently flat at high depth, could automate the identification process and yield a significantly larger set of analysed cases. Positions with eight or more pieces are of particular interest, since they fall above the tablebase boundary and represent the hardest cases for both detection and verification.

Richer evaluation functions offer a second direction. A fortress-aware evaluation function would need to recognise defensive configurations that are structurally self-sustaining and scale the attacking side’s evaluation accordingly. Neural network-based engines such as Leela Chess Zero learn positional features implicitly from game data rather than from hand-coded heuristics, and whether they perform better than Stockfish on fortress positions is an open empirical question worth investigating.

Finally, the breakthrough search component of this engine could be refined and, in principle, integrated into an existing engine as a post-processing step. After completing its standard search, an engine could apply rising evaluation analysis to identify positions where its output is likely unreliable, and flag these to the user rather than presenting a confident but incorrect assessment. This would be a practically valuable improvement even without changes to the underlying evaluation function, and represents a natural next step for a future project to pursue.

Bibliography

- [1] V. Allis, “A knowledge-based approach of connect-four,” Ph.D. dissertation, University of Limburg, 1988.
- [2] W. contributors, *Shannon number*, https://en.wikipedia.org/wiki/Shannon_number, Accessed 2026, 2026.
- [3] H. J. van den Herik, J. W. H. M. Uiterwijk, and J. van Rijswijck, “Games solved: Now and in the future,” *Artificial Intelligence*, vol. 134, no. 1–2, pp. 277–311, 2002.
- [4] BCS, *Computer chess: A historical perspective*, <https://www.bcs.org/articles-opinion-and-research/computer-chess-a-historical-perspective/>, Accessed 2026, 2026.
- [5] W. contributors, *Endgame theory*, https://en.wikipedia.org/wiki/Endgame_theory, Accessed 2026, 2026.
- [6] W. contributors, *Endgame technique*, https://en.wikipedia.org/wiki/Endgame_technique, Accessed 2026, 2026.
- [7] N. D. Elkies, “On numbers and endgames: Combinatorial game theory in chess endgames,” *arXiv*, 1999. [Online]. Available: <https://arxiv.org/abs/math/9905198>
- [8] ChessBio, *Tablebase – endgame database*, <https://chessbio.com/term/tablebase>, Accessed 2026, 2026.
- [9] W. contributors, *Endgame tablebase*, https://en.wikipedia.org/wiki/Endgame_tablebase, Accessed 2026, 2026.
- [10] C. D. Peterson, “The effect of endgame tablebases on modern chess engines,” California Polytechnic State University, Tech. Rep., 2018, Undergraduate research report. [Online]. Available: <https://digitalcommons.calpoly.edu/cpesp/271>
- [11] C. E. Shannon, “Programming a computer for playing chess,” *Philosophical Magazine*, vol. 41, no. 314, pp. 256–275, 1950.
- [12] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.
- [13] D. E. Knuth and R. W. Moore, “An analysis of alpha-beta pruning,” *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.

-
- [14] C. P. Wiki, *Alpha-beta*, <https://www.chessprogramming.org/Alpha-Beta>, Accessed 2026, 2026.
 - [15] C. P. Wiki, *Search*, <https://www.chessprogramming.org/Search>, Accessed 2026, 2026.
 - [16] C. P. Wiki, *Evaluation function*, https://www.chessprogramming.org/Evaluation_Function, Accessed 2026, 2026.
 - [17] C. P. Wiki, *Fortress*, <https://www.chessprogramming.org/Fortress>, Accessed 2026, 2026.
 - [18] J. Mendez et al., “Metamorphic testing of chess engines,” *Information and Software Technology*, vol. 162, p. 107 263, 2023.
 - [19] J. Mendez et al., “Re-evaluating metamorphic testing of chess engines: A replication study,” *Information and Software Technology*, vol. 181, p. 107 679, 2025.
 - [20] C. P. Wiki, *Horizon effect*, https://www.chessprogramming.org/Horizon_Effect, Accessed 2026, 2026.
 - [21] Guid and Bratko, *Detecting fortresses in chess*, <https://ev.fe.uni-lj.si/1-2-2012/Guid.pdf>, Accessed 2026, 2012.
 - [22] W. contributors, *Solving chess*, https://en.wikipedia.org/wiki/Solving_chess, Accessed 2026, 2026.
 - [23] W. contributors, *Forsyth-edwards notation*, https://https://en.wikipedia.org/wiki/Forsyth-Edwards_Notation, Accessed 2026, 2026.
 - [24] W. contributors, *Portable game notation*, https://en.wikipedia.org/wiki/Portable_Game_Notation, Accessed 2026, 2026.
 - [25] C. P. Wiki, *Universal chess interface (uci)*, <https://www.chessprogramming.org/UCI>, Accessed 2026, 2026.
 - [26] C. P. Wiki, *Syzygy tablebases*, https://www.chessprogramming.org/Syzygy_Tablebases, Accessed 2026, 2026.
 - [27] C. P. Wiki, *Testing chess engines*, <https://www.chessprogramming.org/Testing>, Accessed 2026, 2026.